PORTAFOLIO WEB · INGENIERÍA WEB · SEGUNDO PREVIO

BLACKDOT.

Manual técnico de auditoría e infraestructura

Colección Arcade de seis videojuegos web, con Atari Football 2D y Pixel Pong 2D en HTML5 Canvas y JavaScript ES6+: Game Loop, colisiones, IA, persistencia con LocalStorage y despliegue serverless.

AUTORA
Danna Rueda

VERSIÓN
2.1

FECHA
30 de octubre de 2026

URL PÚBLICA
dannavalentinar11-beep.github.io/blackdot-portafolio

92/92
pruebas automatizadas aprobadas

99
Lighthouse móvil · Performance (build local)

6
videojuegos en el Arcade

00 / ÍNDICE

Contenido y control documental

1. Resumen y cumplimiento del previo
2. Arquitectura del proyecto
3. Los seis videojuegos
4. Atari Football 2D (futbol.html)
5. Pixel Pong 2D (pingpong.html)
6. Diagrama de flujo del Game Loop
7. requestAnimationFrame y control del tiempo
8. Detección de colisiones y goles
9. Arquitectura de funciones JavaScript
10. LocalStorage y persistencia
11. Controles de cada videojuego
12. Defectos encontrados y corregidos
13. Evidencias de pruebas funcionales
14. Evidencias visuales
15. GitHub y despliegue HTTPS
16. Informe Lighthouse móvil
17. Resultados de rendimiento
18. Preparación del live coding
19. Conclusiones técnicas

Control documental

CAMPO	VALOR
Documento	Manual técnico de auditoría e infraestructura
Formato	PDF horizontal 1920 × 1080 px
Versión	2.1 (sustituye a la 2.0 de cuatro y seis juegos)
Responsable	Danna Rueda
Asignatura	Ingeniería Web · Segundo Previo
Pruebas ejecutadas	30 de octubre de 2026 a las 1:36 a. m.
Herramientas de verificación	Chrome headless + Puppeteer, Lighthouse 12.3.0

Criterio de evidencia. Todos los resultados de este manual salen de pruebas ejecutadas sobre el código entregado: los JSON de resultados y los informes Lighthouse HTML van en la carpeta `evidencias/` del ZIP. Lo que no pudo medirse se marca como **PENDIENTE**.

01 / RESUMEN

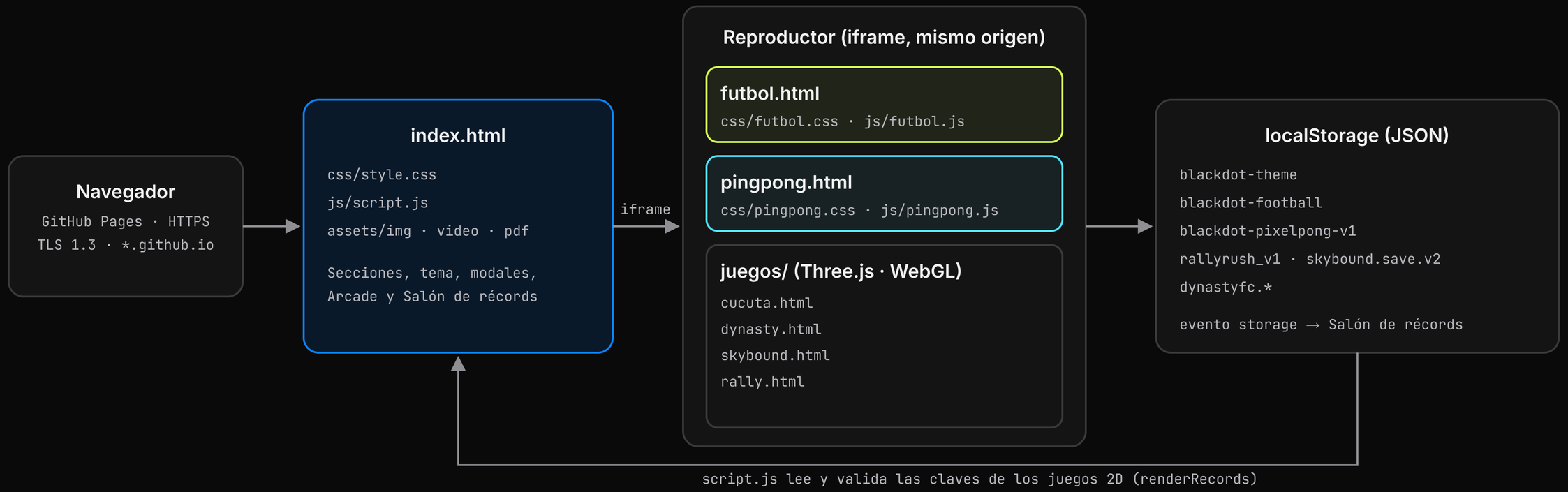
Resumen y matriz de cumplimiento del previo

La auditoría encontró defectos graves en el fútbol original, entre ellos **el partido se congelaba después de cada gol** y una salida por la banda superior contaba como gol, además de un `index.html` de 6,2 MB que dejaba Lighthouse móvil en 55. Todos se corrigieron sin reconstruir los juegos ni cambiar el diseño de BLACKDOT.

REQUISITO DEL PREVIO	ESTADO	EVIDENCIA
Fútbol 2D Atari en `futbol.html`: cancha, arcos, jugadores y arqueros	Cumple	Cancha 1000×610 en Canvas, 2 arcos con red y postes, 6 jugadores de campo y 2 porteros (nuevo)
Bucle a 60 FPS con requestAnimationFrame	Cumple	Un único bucle; 1.52 ms de CPU por frame con CPU ×4 (presupuesto 16,7 ms); 0 tareas largas
Velocidad progresiva	Cumple	speedFactor(): hasta +35 % por tiempo y +15 % por goles; visible en el HUD (VEL ×1.00…)
Conteo de goles en tiempo real	Cumple	Marcador en la barra superior, actualizado en cada frame
Detección de colisiones exacta	Cumple	Postes circulares, bandas, línea de fondo y red; sub-pasos anti-túnel (10 pruebas)
Modo 1 jugador vs IA (seguimiento autónomo del balón)	Cumple	CPU persigue el balón; defensas escalonados; porteros automáticos
Modo 2 jugadores local (P1 WASD · P2 flechas)	Cumple	Controles independientes por jugador (antes se mezclaban al chutar)
Ping-Pong integrado en el Arcade	Cumple	pingpong.html en el reproductor del portafolio, con historial y récords
Integración visual, navegación, modo oscuro/claro y responsive	Cumple	Probado a 375, 768 y 1440 px sin scroll horizontal; tema persistente
Historial de partidos y High Scores en JSON (LocalStorage)	Cumple	Historial con fecha, modo y marcador; Salón de récords; botón para reiniciar estadísticas
Controles por teclado, instrucciones visibles y pausa	Cumple	Instrucciones en el menú y en el HUD; pausa con ESC, P o botón II PAUSA
Despliegue serverless con HTTPS	Parcial	La URL actual responde por HTTPS (TLS 1.3). **PENDIENTE:** publicar la versión 2.1
Manual PDF 1920×1080 con diagramas y arquitectura	Cumple	Este documento
Lighthouse móvil ≥ 85	Cumple (local)	99/100/100/100 en la build local. **PENDIENTE:** medir sobre la URL pública tras publicar
Evidencia Git/GitHub	Pendiente	Requiere capturas del repositorio y de Actions/Pages tras el push (sección 15)

02 / ARQUITECTURA

Arquitectura del proyecto



Antes (v2.0)

Los seis juegos iban dentro de `index.html` como bloques GZIP+Base64, junto con imágenes y videos en `data:`. Resultado: **6.2 MB** de HTML que había que descargar y analizar antes de mostrar nada.

Ahora (v2.1)

Cada juego es su propio archivo y se carga solo al pulsar "Jugar". `index.html` pesa **53 KB** y las imágenes están en WebP con `srcset`. El reproductor usa `iframe.src` del mismo origen, así que LocalStorage se comparte con el portafolio.

Árbol de archivos

```
index.html · futbol.html · pingpong.html
css/  style · futbol · pingpong
js/   script · futbol · pingpong
juegos/ cucuta · dynasty · skybound · rally
assets/ img · video · manuales PDF
live-coding/ turbo · viento · obstaculos · gol-gana
```

03 / CATÁLOGO

Los seis videojuegos del Arcade



GAME 01 · Mundo abierto · Acción

Cúcuta Open World

Exploración de Cúcuta: personaje, conducción, radio y actividades.

Three.js / WebGL

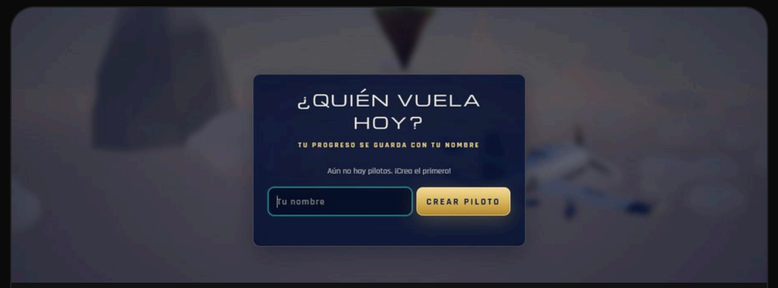


GAME 02 · Deportes · Fútbol 3D

Dynasty FC

Partido, modo carrera, fichajes, repeticiones y celebraciones.

Three.js / WebGL

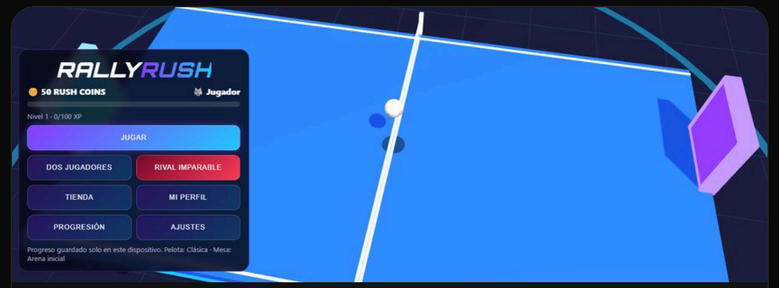


GAME 03 · Aviación · Arcade

Sky Bound

Vuelo arcade con aros, monedas, hangar y batallas aéreas.

Three.js / WebGL



GAME 04 · Ping-pong 3D

Rally Rush

Paletas 3D, IA por niveles, 1 o 2 jugadores, tienda y progreso.

Three.js / WebGL

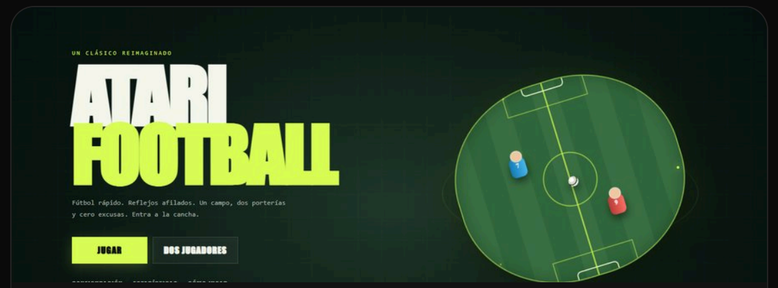


GAME 05 · Arcade · Tenis de mesa

Pixel Pong 2D

Contra IA, dos jugadores y desafío; velocidad creciente.

Canvas 2D · ES6+



GAME 06 · Deportes · Fútbol 2D

Atari Football 2D

CPU o 2 jugadores, porteros, velocidad progresiva y récords.

Canvas 2D · ES6+

Capturas tomadas por la prueba automatizada al abrir cada tarjeta del Arcade en el reproductor del portafolio (sección 13). Los cuatro juegos 3D se conservan sin cambios en su código; solo se movieron a `juegos/`.

04 / FÚTBOL

Atari Football 2D · futbol.html



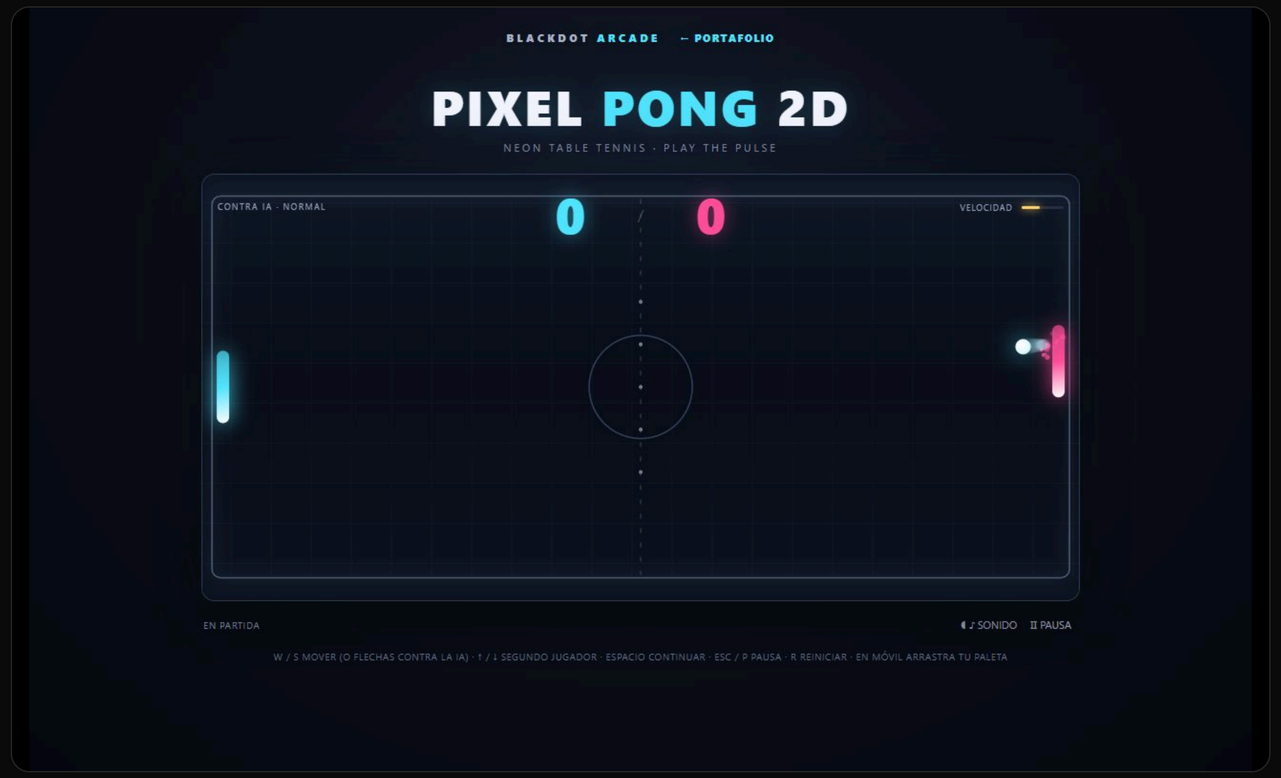
Partido 1 jugador vs CPU: porteros con dorsal 1 (amarillo y naranja), marcador, reloj y velocidad progresiva.

Elementos del juego

- **Cancha** 852×474 dentro de un canvas de 1000×610, con áreas, círculo central, público y focos.
- **Arcos** con red de 22 px de profundidad y **postes con colisión circular**.
- **8 futbolistas:** J1, dos apoyos azules, delantero rojo (CPU o J2), dos apoyos rojos y **2 porteros automáticos**.
- **Física del balón:** velocidad, rozamiento (×0,988 por frame a 60 Hz), efecto de giro, estela y sub-pasos.
- **Chute con carga:** mantener ESPACIO o ENTER aumenta la potencia de 360 a 750 px/s, multiplicada por la velocidad progresiva.
- **Modos:** 1 jugador vs CPU (4 dificultades) y 2 jugadores local.
- **Fin del partido:** al agotarse el tiempo (90 s a 5 min) o al llegar al límite de goles (3 a 10).
- **Pausa:** ESC, P o botón II PAUSA; también automática al cambiar de pestaña.
- **Estadísticas:** partidos, victorias, empates, derrotas, goles, mejor racha, récord de goles e historial de los últimos 10 partidos.

05 / PING-PONG

Pixel Pong 2D · pingpong.html



Pixel Pong 2D contra la IA: marcador neón, modo de juego y barra de velocidad.

Elementos del juego

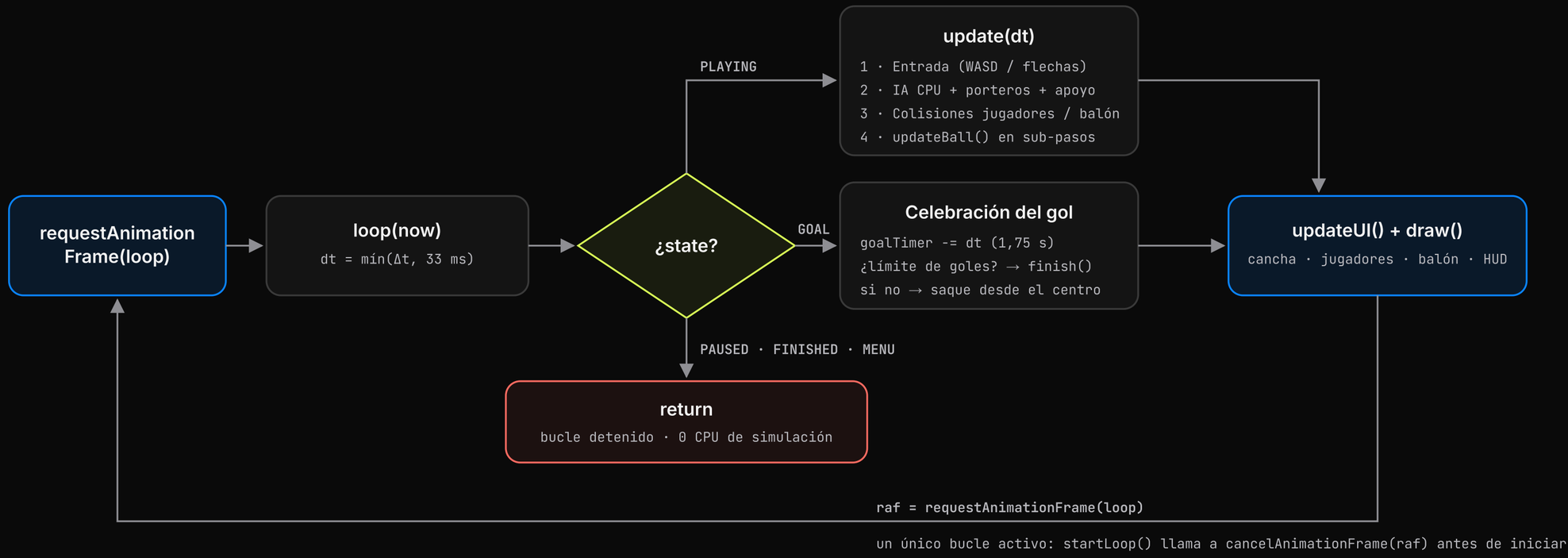
- **Modos:** contra IA (4 dificultades, objetivo de 5 a 15 puntos), dos jugadores local y desafío arcade.
- **Física:** sub-pasos según la velocidad, rebote en bandas y ángulo de salida según el punto de impacto en la paleta.
- **Velocidad progresiva:** cada golpe acelera la pelota un 4,5 % (con tope); barra VELOCIDAD en el HUD.
- **IA:** predice el punto de llegada con los rebotes y aplica un error según la dificultad.
- **Táctil:** cada dedo controla la paleta de su mitad (multitáctil en modo local).
- **Pausa:** ESC, P o botón; ESPACIO continúa; R reinicia.
- **Correcciones v2.1:** historial sin duplicados, flechas para J1 contra la IA, un solo AudioContext, datos dañados tolerados y reinicio de estadísticas.



Modo dos jugadores (W/S y flechas).

06 / GAME LOOP

Diagrama de flujo del Game Loop



Un solo bucle

`startLoop()` llama a `cancelAnimationFrame(raf)` antes de pedir un frame nuevo. En la versión original, R durante el partido lanzaba bucles en paralelo (prueba con **724 rAF/s**).

Estados

`MENU` → `PLAYING` ⇄ `PAUSED`, `PLAYING` → `GOAL` → `PLAYING`, → `FINISHED`. En el original, el estado GOAL hacía salir del bucle y **la pantalla se congelaba tras cada gol**.

Pausa sin consumo

En PAUSED, FINISHED o MENU el bucle no se vuelve a programar: **0 rAF/s** medidos en pausa. Al reanudar, `last` se reinicia para que dt no dé un salto.

07 / rAF

requestAnimationFrame y control del tiempo

```
function startLoop() {
  cancelAnimationFrame(raf);          // nunca dos bucles
  last = performance.now();
  raf = requestAnimationFrame(loop);
}

function loop(now) {
  const dt = clamp((now - last) / 1000, 0, 0.033);
  last = now;
  if (state === 'PLAYING') update(dt);
  else if (state === 'GOAL') { /* celebración */ }
  else return;                        // pausa: el bucle se detiene
  updateUI();
  draw();
  raf = requestAnimationFrame(loop);
}
```

¿Por qué requestAnimationFrame?

- Se sincroniza con el refresco de la pantalla (60 Hz o más) y con el compositor del navegador.
- Se pausa solo en pestañas ocultas; además el juego pasa a PAUSED con `visibilitychange` y `blur`.
- No acumula llamadas como `setInterval` cuando un frame tarda.

Independencia de los FPS

Todo movimiento se multiplica por `dt`. Durante la auditoría se encontró que la IA del original usaba `react·dt·3` como ganancia, lo que la hacía hasta 3 veces más lenta en monitores de 144–180 Hz. Ahora usa una ganancia constante equivalente a 60 Hz.

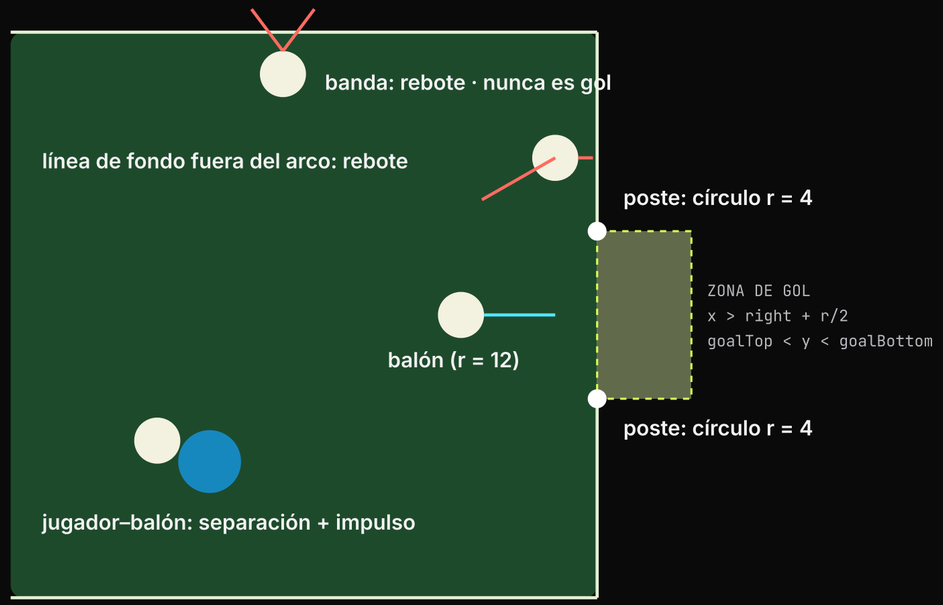
Medición

JUEGO	CPU	FPS MEDIDOS*	MS CPU POR CALLBACK RAF**	TAREAS LARGAS
futbol.html	normal	178	0.42	0
futbol.html	ralentizada ×4	177.2	1.52	0
pingpong.html	normal	172.7	0.1	0
pingpong.html	ralentizada ×4	168.6	0.39	0

* Chrome headless no limita a 60 Hz, así que los FPS medidos muestran la capacidad máxima. El dato clave es el coste por frame: **≤ 1.52 ms** frente al presupuesto de 16,7 ms para 60 FPS, incluso con la CPU ralentizada ×4.
** Pixel Pong usa dos callbacks por frame (update y render): su coste por frame es el doble del valor indicado.

08 / COLISIONES

Detección de colisiones y goles



Canvas interno 1000×610 · FIELD.left = 74 · right = 926 · top = 68 · bottom = 542
goalTop = 260 · goalBottom = 350 · red de 22 px de profundidad

Orden en cada frame

1. `collidePlayers()`: círculo contra círculo entre los 8 jugadores. Separación por solapamiento e intercambio de velocidad normal (×0,55).
2. `collideBallPlayers()`: saca el balón al borde del jugador y le suma un impulso más el 72 % de la velocidad del jugador.
3. `updateBall(dt)`: integra el balón en **sub-pasos** (n = ⌈v·dt / (r/2)⌉), así que nunca avanza más de 6 px por sub-paso.
4. En cada sub-paso, `collideField()` revisa bandas, línea de fondo, postes (reflexión `v - 1,75(v·n)n`), red y gol.

Regla de gol

Hay gol si `goalTop < y < goalBottom` y el centro supera la línea en más de medio radio (`x < left - r/2` o `x > right + r/2`). Fuera de la boca, la línea de fondo rebota. **Las bandas nunca son gol** (en el original sí lo eran entre x=444 y x=556).

Pruebas de precisión

CASO	RESULTADO
IA (CPU) persigue el balón de forma autónoma	✓ Aprobada
Gol en el arco derecho suma al AZUL	✓ Aprobada
Gol en el arco izquierdo suma al ROJO	✓ Aprobada
Salida por banda superior rebota y no es gol	✓ Aprobada
Rebote en la línea de fondo fuera del arco (no es gol)	✓ Aprobada
Rebote en el poste superior del arco derecho	✓ Aprobada
Gol en la esquina superior de la boca (roza el poste y entra)	✓ Aprobada
Gol en la esquina inferior de la boca	✓ Aprobada
Balón que impacta de lleno el poste inferior rebota (no es gol)	✓ Aprobada
Balón muy rápido no atraviesa el arco sin registrar gol (sub-pasos)	✓ Aprobada
Tras un gol se reanuda el juego con saque desde el centro	✓ Aprobada

09 / FUNCIONES

Arquitectura de funciones JavaScript

1 · ESTADO Y PERSISTENCIA

FIELD · POSTS
loadStats()
save()
speedFactor()

2 · MECÁNICAS (live coding)

Mecanicas.registrar(m)
Mecanicas.emitir(evento)
Mecanicas.factor(evento)
window.Futbol

3 · INTERFAZ (DOM)

chooseStart(modo)
instructions()
statsScreen()
settingsScreen()

4 · ENTRADA

keydown / keyup
readInput(controles)
pad táctil
blur / visibilitychange

5 · JUGADORES E IA

movePlayer(p, ix, iy, dt)
aiUpdate(dt)
keeperUpdate(p, dt)
supportUpdate(dt) · kick()

6 · FÍSICA Y COLISIONES

collidePlayers()
collideBallPlayers()
collidePost(poste)
collideField() · updateBall()

7 · GOLES Y ESTADÍSTICAS

scoreGoal(lado)
finish()
updateUI()
historial JSON

8 · GAME LOOP

startLoop() · loop(now)
update(dt)
togglePause()
startGame() · restart()

9 · RENDER CANVAS 2D

draw()
drawField()
drawPlayer(p)
drawBall() · HUD/FPS

js/futbol.js

IIFE en modo estricto con 9 bloques numerados en el propio archivo. Expone solo `window.Futbol = { Mecanicas, juego }` para el live coding, y `__AF` únicamente con `?qa=1`.

js/pingpong.js

Dos bucles rAF (update y render) del diseño original, `loadData()` con validación, un solo `AudioContext`, control táctil por `pointerId` y `__PP` solo con `?qa=1`.

js/script.js

Portafolio en vanilla JS: tema, cursor, menú, modales, proyectos, blog, formulario, Arcade (`gameCard`, `playGame`, `mountFrame`, `closePlayer`) y Salón de récords (`readRecord`, `renderRecords`, `initRecords`).

10 / PERSISTENCIA

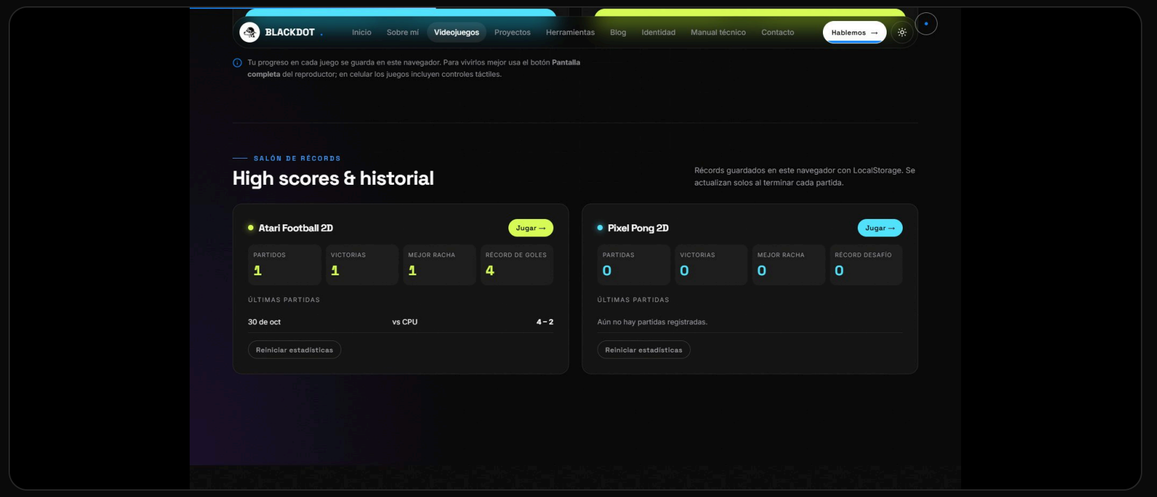
LocalStorage y persistencia

CLAVE	CONTENIDO (JSON)	LO USA
blackdot-theme	"light" | "dark"	Portafolio
blackdot-football	played, won, drawn, lost, scored, conceded, streak, best, bestGoals, history[ ], sound, difficulty	Atari Football · Salón de récords
blackdot-pixelpong-v1	games, wins, losses, points, best, streak, bestStreak, history[ ], sound, difficulty	Pixel Pong · Salón de récords
rallyrush_v1 · skybound.save.v2 · dynastyfc.*	Progreso propio de cada juego 3D	Juegos 3D (sin cambios)

```
// Escritura
localStorage.setItem('blackdot-football', JSON.stringify(stats));
// Historial (últimos 10)
{ "date": "2026-10-30T05:54:02.409Z", "mode": "two",
  "a": 3, "b": 1, "duration": 120 }
```

Robustez

- `loadStats()` y `loadData()` validan cada campo: JSON roto, `null`, arrays o tipos incorrectos vuelven a los valores por defecto.
- Los datos con el formato anterior (`recent: ["2-1"]`) se migran solos a `history`.
- **Reiniciar estadísticas** desde el juego o desde el portafolio, conservando sonido y dificultad.
- El portafolio escucha el evento `storage` y refresca el Salón de récords al terminar cada partido.



Salón de récords del Arcade: high scores e historial leídos de LocalStorage, con botón de reinicio.

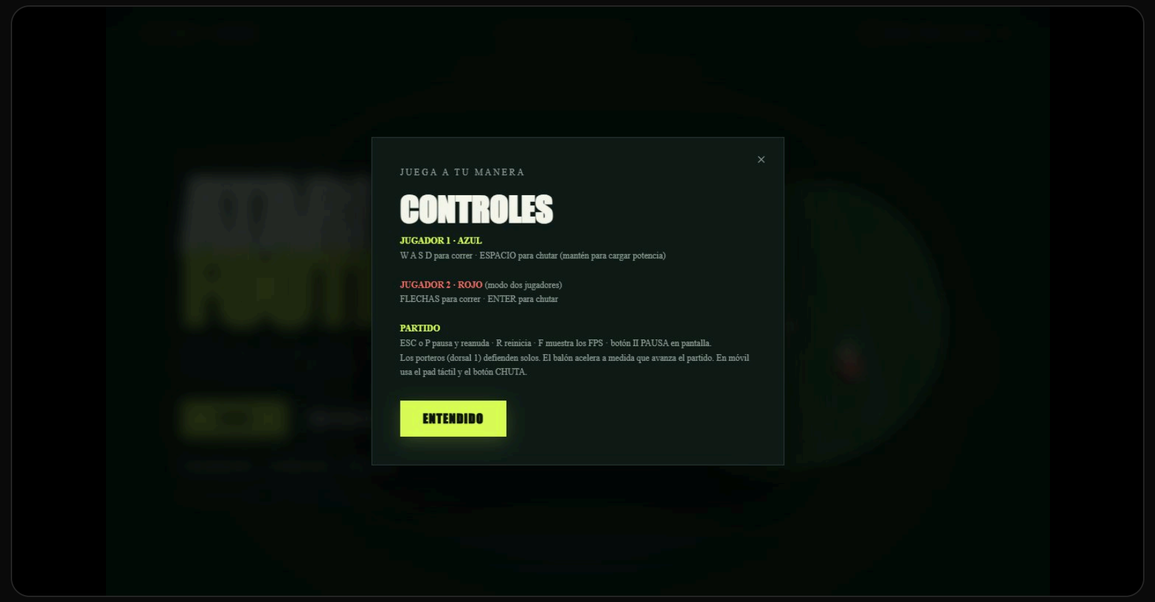


Estadísticas e historial dentro de Atari Football 2D.

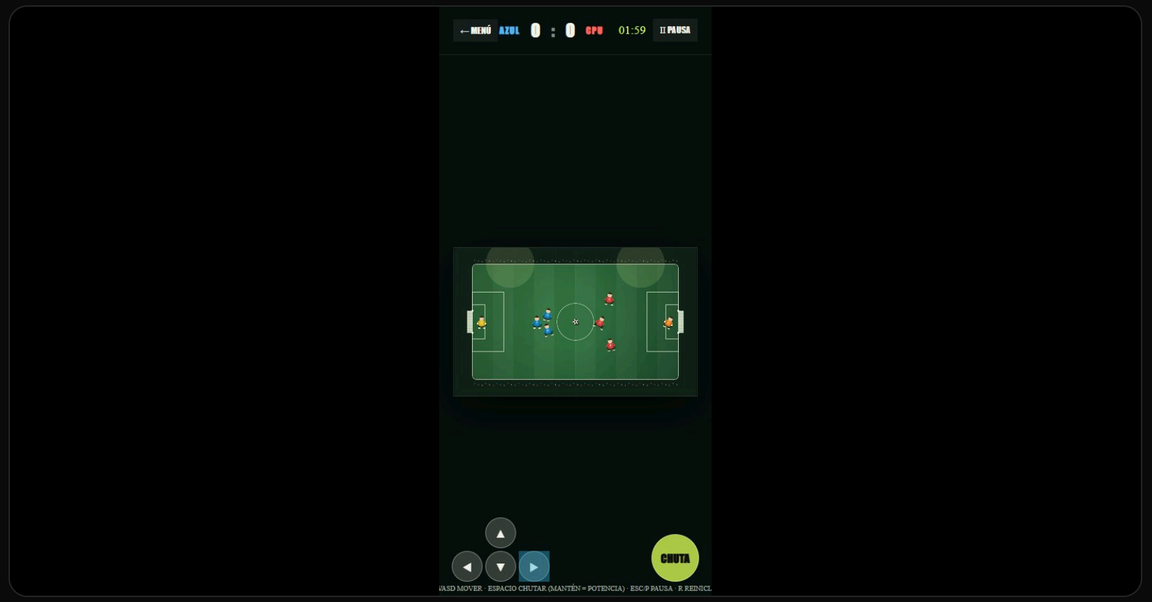
11 / CONTROLES

Controles de cada videojuego

JUEGO	CONTROLES
Cúcuta Open World	W A S D caminar/conducir · Arrastrar cámara · E subir/bajar del carro · Espacio freno de mano · Clic disparar · M mapa · R emisora · Esc pausa
Dynasty FC	W A S D mover · Espacio/Shift sprint · J pase · K disparo · L pase largo · U entrada · Q/I cambiar jugador · E/H regate · 1–8 celebración · Esc/P pausa
Sky Bound	W A S D o flechas dirigir · Espacio frenar (entrenamiento) · Espacio/J/clic disparar (batalla) · Esc pausa
Rally Rush	Ratón/dedo mover paleta · W/S jugador 1 · ↑/↓ jugador 2 · Esc/P pausa
Pixel Pong 2D	W/S jugador 1 (o flechas contra la IA) · ↑/↓ jugador 2 · Arrastrar en el celular · Espacio continuar · Esc/P pausa · R reiniciar
Atari Football 2D	J1: W A S D + Espacio (mantener = potencia) · J2: flechas + Enter · Esc/P o botón II PAUSA · R reiniciar · F ver FPS · pad táctil + CHUTA en el celular



Instrucciones visibles en Atari Football 2D (menú “Cómo jugar” y franja inferior del partido).



Controles táctiles del fútbol en un celular de 390 px de ancho.

12 / AUDITORÍA

Defectos encontrados y corregidos

Prueba sobre la **versión original**, con una copia que expone el estado interno solo para medir (archivo `qa-original.json`): 11 de 11 comprobaciones fallaron.

COMPROBACIÓN SOBRE EL CÓDIGO ORIGINAL	ORIGINAL	DETALLE MEDIDO	V2.1
F1 porteros en la cancha	✗ Falla	[{"team":0,"x":290,"num":7},{"team":0,"x":385,"num":11},{"team":0,"x":385,"num":10},{"team":1,"x":710,"num":9},{"team":1	✓ Corregido
F2 salida por banda superior no es gol	✗ Falla	{"score":[0,1],"state":"GOAL"}	✓ Corregido
F7 el partido se reanuda después de un gol (no se congela)	✗ Falla	estado 2,5 s después del gol = GOAL	✓ Corregido
F3 reiniciar (R) no duplica el bucle rAF	✗ Falla	rAF/s antes=0 después de 4×R=724	✓ Corregido
F4 empate no se registra como derrota	✗ Falla	{"played":1,"won":0,"lost":1,"recent":["1–1"]}	✓ Corregido
F5 datos corruptos en LocalStorage no rompen Estadísticas	✗ Falla	stats.recent.join is not a function	✓ Corregido
P1 historial: partida local se guarda una sola vez	✗ Falla	["LOCAL 3–2","LOCAL 3–2"]	✓ Corregido
P2 contra IA: flechas mueven al jugador 1 (como indica la ayuda del portafolio)	✗ Falla	y antes=223.0 después=223.0	✓ Corregido
P3 datos parciales en LocalStorage no rompen el fin de partida	✗ Falla	endMatch: Cannot read properties of undefined (reading 'unshift')	✓ Corregido
P4 opción para reiniciar estadísticas	✗ Falla	búsqueda en código	✓ Corregido
F6 opción para reiniciar estadísticas	✗ Falla	búsqueda en código	✓ Corregido

Otros hallazgos

Los compañeros rojos no se movían en modo 2 jugadores; el chute de J1 tomaba la dirección de las flechas de J2; no había porteros; la IA dependía de los FPS.

Portafolio

`index.html` de 6,2 MB (Lighthouse 55); `article` con `role="button"` (aria-allowed-role); textos que mencionaban cuatro juegos.

Pong

Un AudioContext nuevo por sonido; ESC no pausaba entre puntos; el segundo dedo liberaba la paleta del primero.

13 / PRUEBAS

Evidencias de pruebas funcionales (1/4)

Batería automatizada con Puppeteer sobre Chrome (servidor local con gzip, como GitHub Pages). Se usan teclado, ratón y pantalla táctil reales. **80/80** pruebas funcionales y **12/12** de live coding aprobadas en la última ejecución (30 de octubre de 2026 a las 1:36 a. m.). La batería se ejecutó 3 veces seguidas con el mismo resultado; las primeras ejecuciones también sirvieron para afinar casos de prueba mal planteados (por ejemplo, un portero que bloqueaba el tiro de prueba).

ÁREA	PRUEBA	RESULTADO	DATO MEDIDO
Portafolio	Carga sin errores JavaScript	✓ Aprobada	sin errores
Portafolio	Sin recursos rotos (4xx/5xx o fallidos)	✓ Aprobada	todos los recursos responden
Portafolio	Arcade muestra 6 videojuegos	✓ Aprobada	Jugar Cúcuta Open World, Jugar Dynasty FC, Jugar Sky Bound, Jugar Rally Rush, Jugar Pixel Pong 2D, Jugar Atari
Portafolio	Todos los enlaces internos (#) apuntan a una sección existente	✓ Aprobada	10 destinos verificados
Portafolio	Rutas de archivos locales (img, video, pdf, icon) responden 200	✓ Aprobada	19 rutas OK
Portafolio	Los 6 archivos de juego existen	✓ Aprobada	[["futbol.html",200],["pingpong.html",200],["juegos/cucuta.html",200],["juegos/dynasty.html",200],["juegos/sky
Portafolio	Modo claro/oscuro cambia y se recuerda al recargar (LocalStorage blackdot-theme)	✓ Aprobada	antes=auto → clic=light → tras recargar=light
Portafolio	Responsive 375px sin scroll horizontal	✓ Aprobada	scrollWidth - innerWidth = 0px
Portafolio	Responsive 768px sin scroll horizontal	✓ Aprobada	scrollWidth - innerWidth = 0px
Portafolio	Responsive 1440px sin scroll horizontal	✓ Aprobada	scrollWidth - innerWidth = 0px
Portafolio	Menú móvil abre y cierra	✓ Aprobada	is-open=true
Arcade	Juego 1 (cucuta) se abre en el reproductor	✓ Aprobada	{"title":"CÚCUTA OPEN WORLD","canvases":3,"url":"/juegos/cucuta.html"}
Arcade	Juego 2 (dynasty) se abre en el reproductor	✓ Aprobada	{"title":"DYNASTY FC","canvases":1,"url":"/juegos/dynasty.html"}
Arcade	Juego 3 (skybound) se abre en el reproductor	✓ Aprobada	{"title":"SKYBOUND","canvases":1,"url":"/juegos/skybound.html"}
Arcade	Juego 4 (rally) se abre en el reproductor	✓ Aprobada	{"title":"RALLY RUSH","canvases":2,"url":"/juegos/rally.html"}
Arcade	Juego 5 (pixelpong) se abre en el reproductor	✓ Aprobada	{"title":"Pixel Pong 2D — BLACKDOT ARCADE","canvases":1,"url":"/pingpong.html"}
Arcade	Juego 6 (atarifootball) se abre en el reproductor	✓ Aprobada	{"title":"ATARI FOOTBALL 2D — BLACKDOT ARCADE","canvases":1,"url":"/futbol.html"}
Arcade	Cerrar el reproductor elimina el iframe (libera memoria del juego)	✓ Aprobada	player.hidden && sin iframe = true
Atari Football	Instrucciones visibles de controles (WASD, flechas, pausa)	✓ Aprobada	JUEGA A TU MANERA CONTROLES JUGADOR 1 · AZUL W A S D para correr · ESPACIO para chutar (mantén para cargar pot
Atari Football	Inicia el partido desde el menú (modo 1 jugador vs CPU)	✓ Aprobada	{"state":"PLAYING","n":8,"gk":[[0,112],[1,888]]}
Atari Football	Cancha con porteros en ambos arcos	✓ Aprobada	[[0,112],[1,888]]
Atari Football	IA (CPU) persigue el balón de forma autónoma	✓ Aprobada	distancia CPU-balón 153px → mínima 33px en 2 s (llega al balón sin intervención del teclado; x final del balón

13 / PRUEBAS

Evidencias de pruebas funcionales (2/4)

ÁREA	PRUEBA	RESULTADO	DATO MEDIDO
Atari Football	Rendimiento: ~60 FPS con requestAnimationFrame	✓ Aprobada	medido 180.6 FPS en 3 s (contador del juego 180.2)
Atari Football	Jugador 1 se mueve con W A S D	✓ Aprobada	(290,305) → (346,342)
Atari Football	Jugador 2 se mueve con las flechas (modo dos jugadores)	✓ Aprobada	(710,305) → (651,266)
Atari Football	Las flechas no mueven al jugador 1 en modo dos jugadores	✓ Aprobada	x J1 346 → 348
Atari Football	Chute del jugador 1 con ESPACIO (mantener = potencia)	✓ Aprobada	velocidad del balón tras chutar: 459 px/s
Atari Football	Chute del jugador 2 con ENTER hacia el arco azul	✓ Aprobada	v=439 px/s, vx=-436
Atari Football	Gol en el arco derecho suma al AZUL	✓ Aprobada	{"score":[1,0],"state":"GOAL","ball":{"x":933,"y":305,"vx":807,"vy":0}}
Atari Football	Gol en el arco izquierdo suma al ROJO	✓ Aprobada	{"score":[0,1],"state":"GOAL","ball":{"x":67,"y":305,"vx":-807,"vy":0}}
Atari Football	Salida por banda superior rebota y no es gol	✓ Aprobada	{"score":[0,0],"state":"PLAYING","ball":{"x":500,"y":351,"vx":0,"vy":326}}
Atari Football	Rebote en la línea de fondo fuera del arco (no es gol)	✓ Aprobada	{"score":[0,0],"state":"PLAYING","ball":{"x":706,"y":200,"vx":-487,"vy":0}}
Atari Football	Rebote en el poste superior del arco derecho	✓ Aprobada	{"score":[0,0],"state":"PLAYING","ball":{"x":661,"y":260,"vx":-520,"vy":0}}
Atari Football	Gol en la esquina superior de la boca (roza el poste y entra)	✓ Aprobada	{"score":[1,0],"state":"GOAL","ball":{"x":936,"y":285,"vx":665,"vy":440}}
Atari Football	Gol en la esquina inferior de la boca	✓ Aprobada	{"score":[1,0],"state":"GOAL","ball":{"x":932,"y":328,"vx":677,"vy":-431}}
Atari Football	Balón que impacta de lleno el poste inferior rebota (no es gol)	✓ Aprobada	{"score":[0,0],"state":"PLAYING","ball":{"x":700,"y":422,"vx":-446,"vy":147}}
Atari Football	Balón muy rápido no atraviesa el arco sin registrar gol (sub-pasos)	✓ Aprobada	{"score":[1,0],"state":"GOAL","ball":{"x":936,"y":305,"vx":-790,"vy":0}}
Atari Football	Tras un gol se reanuda el juego con saque desde el centro	✓ Aprobada	estado=GOAL → {"state":"PLAYING","ball":[500,305]}
Atari Football	Velocidad progresiva visible en el HUD	✓ Aprobada	VEL ×1.03 → VEL ×1.32
Atari Football	Pausa y reanudación con ESC (el reloj se detiene)	✓ Aprobada	pausa=PAUSED, reloj quieto=true, reanuda=PLAYING
Atari Football	Pausa y reanudación con P (el reloj se detiene)	✓ Aprobada	pausa=PAUSED, reloj quieto=true, reanuda=PLAYING
Atari Football	Pausa con el botón en pantalla	✓ Aprobada	PAUSED / ▶ SEGUIR → PLAYING
Atari Football	Reiniciar (R) varias veces mantiene un único bucle rAF	✓ Aprobada	rAF/s: 180 → 181 tras 4×R
Atari Football	En pausa el bucle se detiene (0 CPU de simulación)	✓ Aprobada	rAF/s en pausa: 0
Atari Football	Estabilidad: 20 s de juego intenso sin crecimiento de memoria ni bloqueo	✓ Aprobada	heap 1.7 MB → 1.8 MB, estela=7, estado=PLAYING
Atari Football	Empate se registra como empate (no como derrota)	✓ Aprobada	jugados=2 victorias=1 empates=1 derrotas=0
Atari Football	Historial guarda fecha, modo y marcador en JSON	✓ Aprobada	[{"date":"2026-10-30T06:36:19.279Z","mode":"two","a":3,"b":1,"duration":120},{"date":"2026-10-30T06:36:18.262Z
Atari Football	Las estadísticas persisten al cerrar y volver a abrir	✓ Aprobada	{"played":2,"history":2,"best":3}
Atari Football	Botón REINICIAR ESTADÍSTICAS borra estadísticas e historial	✓ Aprobada	{"played":0,"history":[]}
Atari Football	Datos dañados en LocalStorage no rompen el juego: {roto	✓ Aprobada	sin errores, se restablecen valores por defecto

13 / PRUEBAS

Evidencias de pruebas funcionales (3/4)

ÁREA	PRUEBA	RESULTADO	DATO MEDIDO
Atari Football	Datos dañados en LocalStorage no rompen el juego: {"recent":"x","played":"abc","history":5}	✓ Aprobada	sin errores, se restablecen valores por defecto
Atari Football	Datos dañados en LocalStorage no rompen el juego: null	✓ Aprobada	sin errores, se restablecen valores por defecto
Atari Football	Datos dañados en LocalStorage no rompen el juego: [1,2]	✓ Aprobada	sin errores, se restablecen valores por defecto
Atari Football	Migra datos guardados por la versión anterior (recent → history)	✓ Aprobada	[{"date":null,"mode":"ai","a":2,"b":1,"duration":null},{"date":null,"mode":"ai","a":0,"b":3,"duration":null}]
Atari Football	Sin errores de consola durante las pruebas de fútbol	✓ Aprobada	sin errores
Atari Football	Controles táctiles visibles y funcionales en móvil	✓ Aprobada	pad=flex, x 290 → 337
Pixel Pong	Instrucciones de controles visibles	✓ Aprobada	W / S MOVER (O FLECHAS CONTRA LA IA) · ↑ / ↓ SEGUNDO JUGADOR · ESPACIO CONTINUAR · ESC / P PAUSA · R REINICIAR
Pixel Pong	Inicia partida contra IA desde el menú	✓ Aprobada	PLAYING / ai
Pixel Pong	Paleta 1 sube con W	✓ Aprobada	223 → 75
Pixel Pong	Contra la IA, la paleta 1 también responde a las flechas	✓ Aprobada	223 → 366
Pixel Pong	La IA mueve su paleta hacia la trayectoria de la pelota	✓ Aprobada	paleta IA y=332, pelota y=400
Pixel Pong	Colisión pelota–paleta: rebota y acelera	✓ Aprobada	vx=426 (antes -400), marcador 0-0
Pixel Pong	Rebote en la banda superior	✓ Aprobada	vy tras rebote = 400
Pixel Pong	Punto para el rival cuando la pelota pasa la paleta	✓ Aprobada	marcador 0-1
Pixel Pong	Pausa (ESC) y continuar (ESPACIO)	✓ Aprobada	PAUSED → PLAYING
Pixel Pong	Pausa con botón en pantalla y CONTINUAR	✓ Aprobada	PAUSED
Pixel Pong	Reiniciar con R pone el marcador a 0	✓ Aprobada	0-0
Pixel Pong	Modo dos jugadores: flechas mueven la paleta 2	✓ Aprobada	223 → 77
Pixel Pong	Partida local se guarda una sola vez en el historial	✓ Aprobada	[{"date":"2026-10-30T06:36:31.801Z","mode":"local","a":6,"b":4}]
Pixel Pong	Estadísticas persisten al recargar	✓ Aprobada	{"games":2,"wins":1,"history":2}
Pixel Pong	Reiniciar estadísticas	✓ Aprobada	{"games":0}
Pixel Pong	Datos dañados no rompen el juego: {roto	✓ Aprobada	sin errores
Pixel Pong	Datos dañados no rompen el juego: {"games":2}	✓ Aprobada	sin errores
Pixel Pong	Datos dañados no rompen el juego: {"recent":5,"history":"x"}	✓ Aprobada	sin errores
Pixel Pong	Control táctil: arrastrar mueve la paleta	✓ Aprobada	177 → 131
Pixel Pong	Sin errores de consola durante las pruebas de Pong	✓ Aprobada	sin errores
Arcade + LocalStorage	Salón de récords muestra estado vacío sin datos	✓ Aprobada	2 tarjetas vacías
Arcade + LocalStorage	Fútbol jugable dentro del reproductor del portafolio	✓ Aprobada	{"playing":true,"clock":"01:58"}

13 / PRUEBAS

Evidencias de pruebas funcionales (4/4)

ÁREA	PRUEBA	RESULTADO	DATO MEDIDO
Arcade + LocalStorage	El panel del portafolio se actualiza con el partido guardado (evento storage)	✓ Aprobada	Atari Football 2D Jugar → PARTIDOS 1 VICTORIAS 1 MEJOR RACHA 1 RÉCORD DE GOLES 4 ÚLTIMAS PARTIDAS 30 de oct vs
Arcade + LocalStorage	Reiniciar estadísticas desde el portafolio	✓ Aprobada	Atari Football 2D Jugar → PARTIDOS 0 VICTORIAS 0 MEJOR RACHA 0 RÉCORD DE GOLES 0 ÚLTIMAS PARTIDAS Aún no hay p
Live coding	Turbo: el jugador avanza más con Shift	✓ Aprobada	sin turbo 70 px · con turbo 123 px en 0,6 s
Live coding	Turbo: la energía se agota y vuelve a velocidad normal	✓ Aprobada	tras 2,6 s de turbo avanza 65 px en 0,5 s
Live coding	Turbo sin errores JavaScript	✓ Aprobada	sin errores
Live coding	Viento: desvía la trayectoria de un balón lanzado en línea recta	✓ Aprobada	vy inicial 0 → -18.8 px/s
Live coding	Viento sin errores JavaScript	✓ Aprobada	sin errores
Live coding	Obstáculos: se generan en la cancha	✓ Aprobada	4 obstáculos
Live coding	Obstáculos: el balón rebota y no los atraviesa	✓ Aprobada	vx 500 → -306, dentro=false
Live coding	Obstáculos sin errores JavaScript	✓ Aprobada	sin errores
Live coding	Gol Gana: empate al final del tiempo → muerte súbita (no termina)	✓ Aprobada	estado con reloj en 0 y 1-1: PLAYING
Live coding	Gol Gana: el siguiente gol termina el partido	✓ Aprobada	{"state":"FINISHED","score":[2,1]}
Live coding	Gol Gana sin errores JavaScript	✓ Aprobada	sin errores
Live coding	Las 4 mecánicas juntas mantienen la fluidez del Game Loop	✓ Aprobada	181 FPS, errores: 0

14 / CAPTURAS

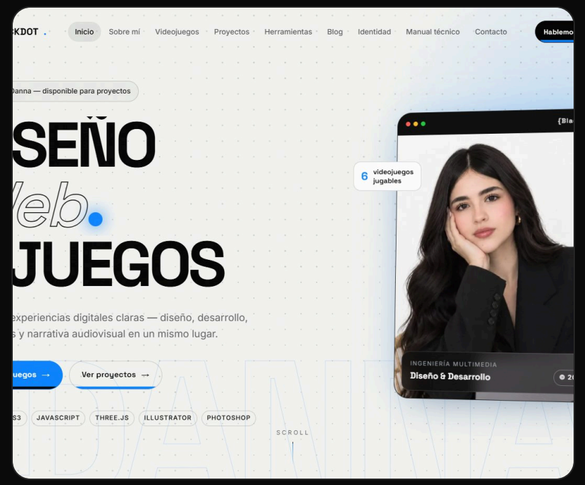
Evidencias visuales



Portafolio a 375 px



Portafolio a 768 px



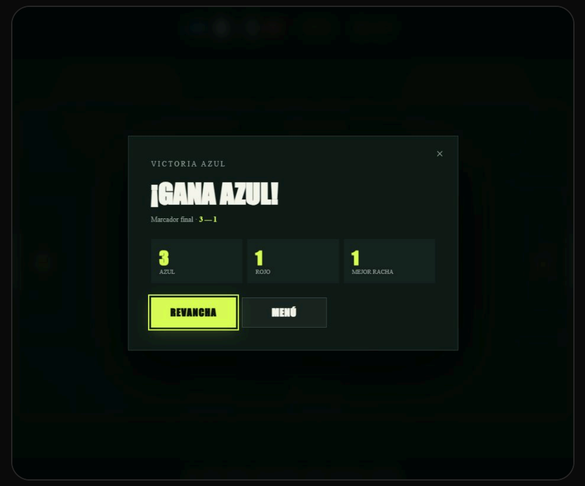
Modo claro persistente



Pausa del fútbol



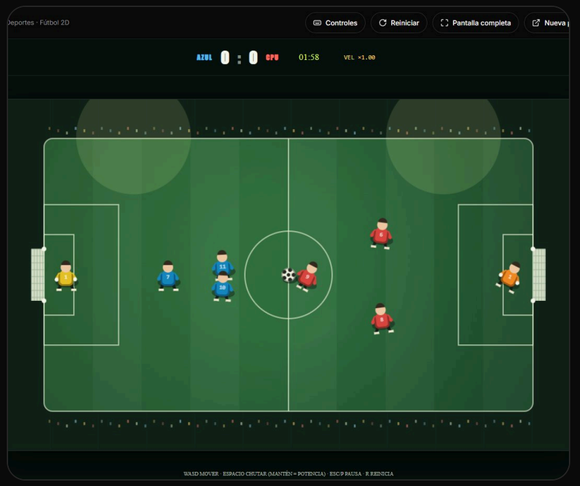
Detección de gol



Fin de partido



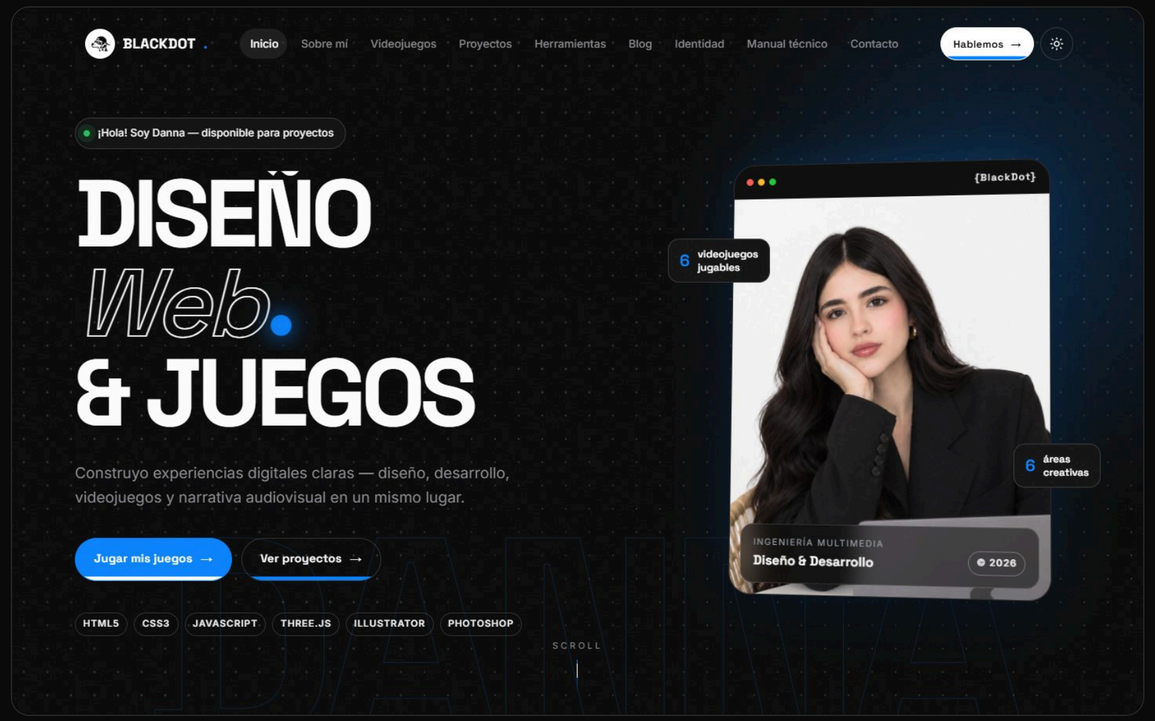
Estadísticas de Pong



Fútbol en el reproductor

15 / DESPLIEGUE

GitHub y despliegue HTTPS



Captura real de la URL pública (30 de octubre de 2026 a las 1:37 a. m.). Todavía sirve la versión anterior.

Verificación real de la URL pública

COMPROBACIÓN	RESULTADO
URL	https://dannavalentinar11-beep.github.io/blackdot-portafolio/
Código HTTP	200
HTTPS	Sí
Protocolo	TLS 1.3
Certificado	*.github.io · emisor YR1
Validez	2026-09-30 → 2026-12-29
Servidor	GitHub.com
Versión publicada	anterior (juegos embebidos en index.html)
index.html publicado	6.23 MB

PENDIENTE: completar después de publicar la v2.1

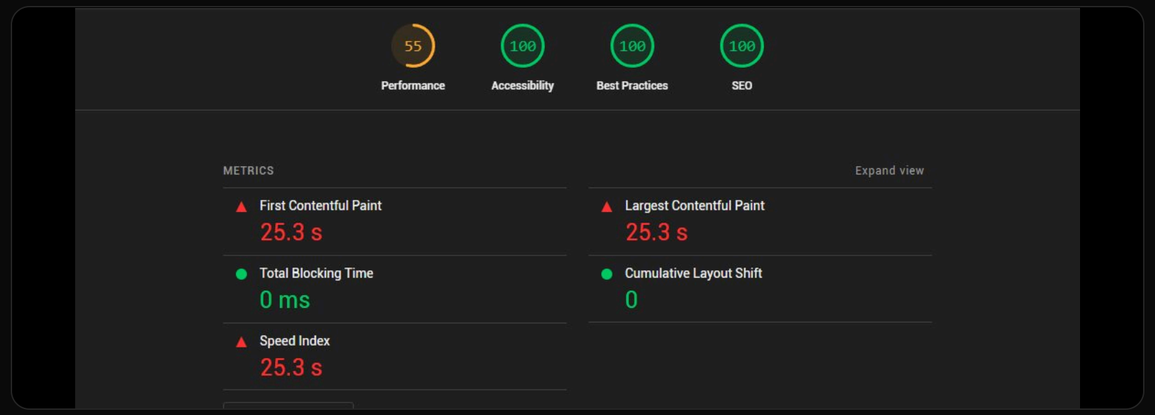
1. Subir el contenido de la carpeta del ZIP a la raíz del repositorio `blackdot-portafolio` (rama `main`).
2. Captura del repositorio: árbol de archivos e historial de commits.
3. Captura de **Settings** → **Pages** (fuente: rama `main`, "Your site is live", Enforce HTTPS) y de la pestaña **Actions** (pages-build-deployment en verde).
4. Abrir la URL en ventana privada y confirmar que carga `futbol.html` y `pingpong.html`.

> No se incluyen capturas del repositorio porque este entorno no tiene acceso a la cuenta de GitHub.

16 / LIGHTHOUSE

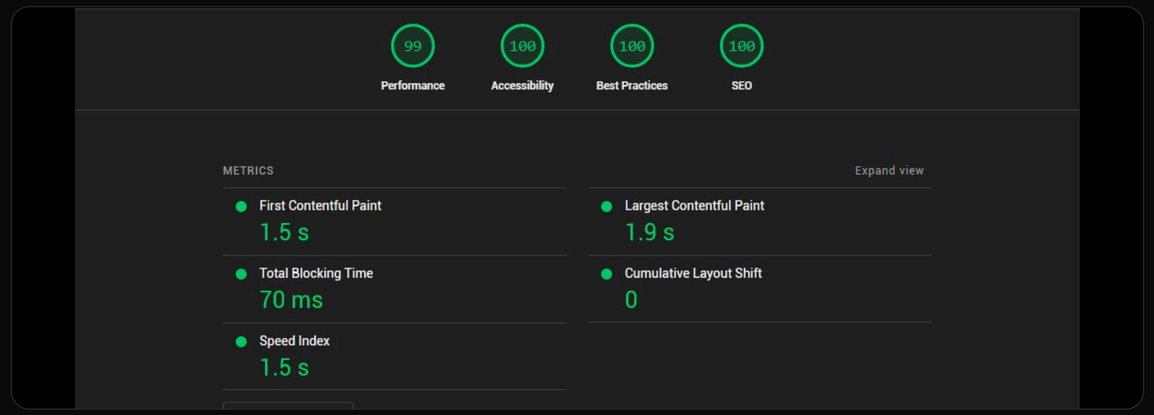
Informe Lighthouse móvil

Antes: URL pública actual (v2.0)



Lighthouse 12.3.0 · móvil · 30 de octubre de 2026 a las 12:36 a. m.

Después: build v2.1 (servidor local con gzip)



Lighthouse 12.3.0 · móvil · 30 de octubre de 2026 a las 1:38 a. m.

MÉTRICA (MÓVIL)	ANTES (URL PÚBLICA)	V2.1 CORRIDA 1	V2.1 CORRIDA 2	V2.1 CORRIDA 3
Performance	55	99	99	96
Accessibility	100	100	100	100
Best Practices	100	100	100	100
SEO	100	100	100	100
First Contentful Paint	25.3 s	1.5 s	1.5 s	1.6 s
Largest Contentful Paint	25.3 s	1.9 s	1.9 s	2.1 s
Total Blocking Time	0 ms	70 ms	80 ms	180 ms
Cumulative Layout Shift	0	0	0	0
Speed Index	25.3 s	1.5 s	1.6 s	1.6 s
Avoid enormous network payloads	Total size was 4,578 KiB	Total size was 250 KiB	Total size was 250 KiB	Total size was 250 KiB

Importante: el ≥ 85 exigido se cumple en la build local con throttling móvil simulado. La medición oficial debe repetirse sobre la URL pública después de publicar (instrucciones en `INSTRUCCIONES-GITHUB-PAGES.md`). Los informes HTML completos van en `evidencias/`.

17 / RENDIMIENTO

Resultados de rendimiento y optimización

6.2 MB → 53 KB
tamaño de index.html

4,578 KiB → 250 KiB
bytes transferidos en la carga inicial

25.3 s → 1.9 s
LCP móvil

OPTIMIZACIÓN	DETALLE	RIESGO CONTROLADO
Juegos fuera de index.html	futbol.html, pingpong.html y juegos/*.html se descargan solo al jugar	Mismo origen: LocalStorage y guardados de los juegos 3D se conservan
Imágenes y videos a archivos	17 referencias data: (16 archivos únicos) → assets/img y assets/video (los videos mantienen preload="none")	Mismas imágenes; rutas verificadas (todas responden 200)
WebP + srcset	Hero 88 → 35 KB (19 KB en móvil); sobre mí 77 → 27/15 KB; pósters −45 a −51 %	Misma imagen y proporción; ancho y alto fijados (CLS 0)
Fuentes sin bloquear el render	Google Fonts con preload + onload y respaldo en noscript	font-display: swap ya estaba activo
Accesibilidad	article role=button → div role=button	Las clases CSS no cambian
Game Loop	cancelAnimationFrame único, pausa sin bucle, AudioContext reutilizado	Heap estable en 20 s de juego intenso (prueba de estabilidad)

JUEGO	CPU	FRAMES EN 10 S	P95 INTERVALO	MS CPU / CALLBACK RAF	TAREAS LARGAS	HEAP INICIO → FIN
futbol.html	normal	1781	5.7 ms	0.42	0	1.33 → 2.04 MB
futbol.html	ralentizada ×4	1772	5.7 ms	1.52	0	1.33 → 2.18 MB
pingpong.html	normal	1727	5.7 ms	0.1	0	1.24 → 2.08 MB
pingpong.html	ralentizada ×4	1687	5.7 ms	0.39	0	1.22 → 1.94 MB

18 / LIVE CODING

Preparación del live coding

Sistema de mecánicas (ganchos)

El motor llama a `Mecanicas.emitir('evento')` en puntos fijos del Game Loop. Una mecánica nueva es un objeto que se registra sin tocar el motor: `alIniciar`, `alActualizar`, `alMoverBalon`, `alColisionBalon`, `velocidadJugador`, `alChutar`, `alGol`, `despuesDeGol`, `alTerminarTiempo`, `alDibujar`, `alDibujarHUD` y `alTecla`.

MECÁNICA	ARCHIVO	GANCHOS	PRUEBA
Turbo con barra de energía	live-coding/turbo.js	alActualizar · velocidadJugador · alDibujarHUD	✓ Aprobada
Viento	live-coding/viento.js	alMoverBalon · alDibujar · alDibujarHUD	✓ Aprobada
Obstáculos aleatorios	live-coding/obstaculos.js	alColisionBalon · alActualizar · alDibujar	✓ Aprobada
Gol Gana / muerte súbita	live-coding/gol-gana.js	alTerminarTiempo · despuesDeGol	✓ Aprobada

> Ninguna mecánica viene activada en el juego entregado. Se activan descomentando una línea en `futbol.html`. La guía `live-coding/GUIA-LIVE-CODING.md` incluye versiones mínimas de 10 a 15 líneas (también probadas), un plan minuto a minuto y respuestas para la sustentación.



Las cuatro mecánicas cargadas a la vez: barra de turbo, indicador y ráfagas de viento, y obstáculos naranjas. Sin errores y sin pérdida de fluidez.

19 / CONCLUSIONES

Conclusiones técnicas

- El Arcade de BLACKDOT integra **seis videojuegos**: los cuatro 3D se conservan intactos y los dos 2D cumplen el previo.
- El fútbol tenía un **defecto excluyente** (congelamiento tras cada gol) que habría llevado el criterio de física a 0,0. Se corrigió reestructurando el Game Loop sin cambiar gráficos ni mecánicas, y quedó cubierto por pruebas.
- La detección de goles ahora es exacta: postes circulares, red, bandas que nunca cuentan como gol y sub-pasos contra el efecto túnel.
- La persistencia en LocalStorage guarda historial y high scores en JSON, tolera datos dañados, migra datos antiguos y permite reiniciar las estadísticas, como pide el nivel Excelente de la rúbrica.
- Sacar los juegos de `index.html` llevó Lighthouse móvil de **55** a **99** (build local), con Accesibilidad, Buenas prácticas y SEO en 100.
- El motor del fútbol quedó legible y modular. Los ganchos permiten implementar la mecánica sorpresa en minutos y explicarla con claridad.
- **Pendiente para cerrar la entrega:** publicar la v2.1 en GitHub Pages, repetir Lighthouse sobre la URL pública y añadir las capturas del repositorio y de Pages.